

Setup outils et environnement

Site de l'AEGP - Guide d'installation

Mai 2026

Rédacteur du guide :

Ilan Guerizec (Promo 2028)

Développeurs :

Ilan Guerizec (Promo 2028)

Marc Bonnin (Promo 2028)

Duane Marchais (Promo 2028)

1. Introduction.....	2
2. Outils à installer.....	2
a. Étape 0 : Vérifier l'installation.....	2
b. Étape 1 : Git.....	2
c. Étape 2 : Node.js (via nvm).....	3
d. Étape 3 : Python 3.14.....	3
e. Étape 4 : VS Code.....	5
3. Récupérer le projet et Workflow git.....	6
a. Cloner le repo (première fois).....	6
e. Workflow quotidien.....	6
f. Travailler sur une branche.....	6
g. Pull request (PR).....	7
h. Gérer un conflit.....	7
4. Comptes en lignes.....	9
5. Variables d'environnement.....	10
6. Outils collaboratifs.....	10

1. Introduction

Le but de ce guide est d'aider les nouveaux/nouvelles admins du site web de l'AEGP à installer les outils nécessaires au développement et à l'entretien du site.
Ce guide est conçu pour Windows (avec WSL2), macOS et Linux.

2. Outils à installer

Outil	Pourquoi	Version
Git	Versionner le code	dernière stable
Node.js	frontend React	20 LTS
Python	backend FastAPI	3.14
VSCode	Éditeur (recommandé)	dernière stable

Pour les utilisateurs Windows :

Installer WSL2 (Windows Subsystem for Linux) et faire tout le développement dedans.

Tutoriel officiel : <https://learn.microsoft.com/fr-fr/windows/wsl/install>

a. Étape 0 : Vérifier l'installation

La première étape est de vérifier ce qui est déjà installé sur votre machine.

Dans un terminal :

```
git --version
node --version
python3 --version
```

Si une commande retourne `command not found`, il faut l'installer.

b. Étape 1 : Git

Linux (Debian/Ubuntu) et Windows (WSL) :

```
sudo apt update && sudo apt install -y git
```

macOS :

Si Homebrew est installé :

```
brew install git
```

Sinon, installer Homebrew d'abord : <https://brew.sh>

Configurer git après installation (une seule fois par personne)

```
git config --global user.name "Votre username"  
git config --global user.email "Votre email"
```

c. Étape 2 : Node.js (via nvm)

nvm permet d'avoir plusieurs versions de Node et de basculer entre elles. Si la version de Node venait à changer, les admins n'ont qu'à faire `nvm install 22` et c'est réglé.

Installer nvm (Linux / macOS / WSL) :

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

Fermer et rouvrir le terminal puis :

```
nvm install 20  
nvm use 20  
nvm alias default 20
```

Vérifier :

```
node --version  #devrait afficher v20.x.x  
npm --version   #devrait afficher 10.x.x
```

d. Étape 3 : Python 3.14

Linux / WSL :

```
#Dépendances de compilation  
sudo apt install -y make build-essential libssl-dev zlib1g-dev  
libbz2-dev \  
libreadline-dev libsqlite3-dev wget curl llvm libncursesw5-dev  
xz-utils \  
tk-dev libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev  
  
#Installer pyenv  
curl https://pyenv.run | bash
```

macOS :

```
brew install pyenv
```

Configurer pyenv dans le shell (à ajouter) ~/.bashrc ou ~/.zshrc) :
Sur macOS avec zsh remplacer "bash" par "zsh".

```
export PYENV_ROOT="$HOME/.pyenv"  
[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"  
eval "$(pyenv init - bash)"
```

Fermer et rouvrir le terminal puis :

```
pyenv install 3.14      #peut prendre un peu de temps  
pyenv global 3.14  
python3 --version      #devrait afficher Python 3.14.x
```

Environnements virtuels Python (venv)

Règle d'or : ne **JAMAIS** installer une lib Python directement avec `pip install` sur le Python global. Sinon le système est pollué et deux projets ne peuvent pas avoir des versions différentes d'une même lib.

Solution : création d'un **environnement virtuel** (un dossier isolé qui contient sa propre version de Python et ses propres libs).

Création de l'environnement virtuel et installation des libs

#Ouvrez le dossier du projet backend et dans la console VS Code :
`python3 -m venv .venv`

#Activer le venv (à refaire à chaque ouverture de VS Code)
#Pour Linux / macOS / WSL
`source .venv/bin/activate`

#Installer les libs à partir du fichier requirements.txt (sur le github)
`pip install -r requirements.txt`

#Pour figer les libs installées dans un fichier (à commiter)
`pip freeze > requirements.txt`

#Désactiver le venv
`deactivate`

Bonnes pratiques :

- Le dossier `.venv/` est dans le `.gitignore`
- Le fichier `requirements.txt` est commité
- Après avoir fait un git pull : réinstaller les libs à partir du `requirements.txt` s'il y a eu changement
- Toujours vérifier que le venv est activé avant de lancer le serveur ou un script (le prompt du terminal affiche `(.venv)` quand c'est actif)

e. Étape 4 : VS Code

Téléchargement : <https://code.visualstudio.com/>

Extensions à installer :

- **ES7+ React/Redux/React-Native snippets** - frontend
- **Tailwind CSS IntelliSense** - frontend
- **Python** (Microsoft) - backend
- **Pylance** (Microsoft) - type checking Python
- **GitLens** - gestion git visuelle
- **Prettier** - formatage
- **Bruno** - tester l'API

Pour Windows WSL : extension **WSL** (Microsoft) pour ouvrir VS Code dans WSL.

3. Récupérer le projet et Workflow git

Le code est hébergé sur le compte GitHub de l'AEGP. Chacun.e des admins doit avoir un **compte personnel Github**.

Les admins des années précédentes ajoutent les nouveaux/nouvelles admins en tant que **collaborateur.ice.s** du repo.

a. Cloner le repo (première fois)

```
git clone https://github.com/AEGPPoitiers/siteofficielAEGP.git
cd siteofficielAEGP/
```

Structure du repo :

- b. frontend/ - application React + Vite
- c. backend/ - API FastAPI
- d. docs/ - documentation (dont ce guide)

e. Workflow quotidien

Avant de commencer à travailler, récupérer les dernières modifs

```
git pull
```

Voir ce qui a été modifié

```
git status
git diff
```

Enregistrer ses modifs en local

```
git add <fichier>      # ou git add . pour tout ajouter
git commit -m "message clair décrivant la modif"
```

Pousser sur github

```
git push
```

f. Travailler sur une branche

On ne pousse pas directement sur la branche “main”. Pour chaque nouvelle fonctionnalité ou correction, on crée une branche :

```
git checkout -b nom-de-la-branche      #Créer + basculer dessus
# modifications + commit ...
git push -u origin nom-de-la-branche   #premier push
git push                                #pushs suivants
```

Convention de nommage : “feat/agenda-vue-mensuelle”, “fix/loginredirect”, “docs/setup-outils”

g. Pull request (PR)

Une fois la branche poussée, sur Github :

- Cliquer sur **Compare & pull request** (proposé automatiquement après un push)
- Titre clair + description (quoi + pourquoi)
- Assigner un.e relecteur.ice de l’équipe
- Après validation : **Merge pull request**, puis **Delete branch** sur Github

Localement, revenir sur **main** et nettoyer :

```
git checkout main
git pull
git branch -d nom-de-la-branche      # supprime la branche locale
```

h. Gérer un conflit

Quand un **git pull** ou un **merge** produit un conflit, Git marque les fichiers concernés. À l’intérieur du fichier on voit :

```
<<<<<<<< HEAD
ma version locale
=====
version distante
>>>>>>>> origin/main
```

Que faire ?

- Ouvrir le fichier, décider quelle version garder (ou combiner les deux)
- Supprimer les ‘<<<<<<<<’, ‘=====’, ‘>>>>>>>>’
- **git add <fichier>** pour marquer le conflit résolu
- **git commit** (ou **git merge -continue**)

Pour annuler le merge et revenir à l'état d'avant :

```
git merge -abort
```

L'extension **GitLens** de VS Code propose une interface visuelle pour gérer les conflits (bouton "Accept Current", "Accept incoming", "Accept both")

4. Comptes en lignes

Voici les comptes créés avec l'adresse mail de l'AEGP :

Service	Pour quoi	Plan
GitHub	Héberger le code, collaboration	Gratuit
Supabase	BDD + Auth + Storage (documents légers seulement)	Free tier
Vercel	Héberger le frontend en prod	Free tier
Render	Héberger le backend FastAPI	Free tier
Blackblaze B2	Stockage des documents du tutorat	Free tier

Les codes d'accès des différents comptes sont détenus par le/la référent.e du bureau de l'AEGP.

Pourquoi deux stockages ?

- **Supabase storage** limité à 1Go sur le plan gratuit donc insuffisant pour les documents du tutorat (estimé à ~10Go pour 5 années de documents).
- **Cloudflare R2** offre 10Go gratuits et 0 frais de bande passante.
- **R2** : documents du tutorat
- **Supabase storage** : documents légers (exemple : affiches des événements)

Limites du plan gratuit Supabase

- Le projet est mis en pause après 7 jours d'inactivité (relance instantanée au prochain accès)

Limites du plan gratuit Render

- Le backend s'endort après 15min d'inactivité (~30s pour se réveiller à la prochaine requête)
- 750h/mois d'exécution

Limites du plan gratuit de Blackblaze

- Bucket privés uniquement, les buckets publics sont payants
- 10Go de stockage
- Les fichiers sont stockés en **bucket privés** et le backend génère à la demande des URLs signées (à durée limitée) que le frontend utilise pour télécharger directement depuis blackblaze

5. Variables d'environnement

Principe :

Les secrets (clés API Supabase, mots de passe, tokens...) ne doivent **JAMAIS** être commités dans Git.

Organisation :

- Chaque projet (front et back) a un fichier .env avec ses variab
- Le fichier .env est dans le .gitignore, il n'est **JAMAIS** commité
- Les vrais valeurs sont partagées par un canal sécurisé

6. Outils collaboratifs

- **Communication** : Serveur Discord
- **Tester l'API** : Bruno (extension VS Code)
- **Explorer la BDD** : interface Web de Supabase